

Linear Regression: Theory + From-Scratch & Library Implementations

Theory

Linear Regression is one of the simplest and most widely used supervised learning algorithms for predicting continuous values. It assumes a **linear relationship** between input features (X) and the target variable (y).

Equation

For one feature: $y = w_0 + w_1x + \epsilon$

For multiple features: $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + \epsilon$

Where:

- w_0 = intercept
- w_i = coefficients (weights)
- ϵ = error term

Goal

Find the best-fit line by minimizing the **Mean Squared Error (MSE)**: $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Solution

Two approaches:

1. **Analytical Solution (Normal Equation)**: $w = (X^T X)^{-1} X^T y$
2. **Iterative Solution (Gradient Descent)**.

Logistic Regression: Theory + From-Scratch & Library Implementations

Theory

Logistic Regression is a supervised learning algorithm used for classification problems (binary or multiclass). Unlike linear regression, it predicts the probability of a class using the sigmoid function:

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

- Output is between 0 and 1
- Decision boundary: If $h_{\theta}(x) \geq 0.5 \rightarrow$ class 1, else class 0

The cost function is the Log Loss:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left(y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right)$$

Optimization is done via Gradient Descent.

Decision Trees: Theory + From-Scratch & Library Implementations

Theory

A Decision Tree is a supervised learning algorithm used for both classification and regression tasks.

- The tree splits the dataset into smaller subsets based on feature conditions.
- Each split is chosen to maximize the purity of child nodes.
- Final predictions come from the leaf nodes.

Key Concepts

- Entropy (measure of impurity):

$$Entropy(S) = - \sum_{c=1}^k p_c \log_2(p_c)$$

where p_c = proportion of class c .

- Information Gain (IG) (used in classification):

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- Gini Impurity (alternative to entropy):

$$Gini(S) = 1 - \sum_{c=1}^k p_c^2$$

- Stopping criteria: max depth, min samples per split, or pure nodes.

Random Forest: Theory + From-Scratch & Library Implementations

Theory

Random Forest is a **supervised ensemble learning algorithm** used for both **classification** and **regression** tasks. It builds multiple **decision trees** and combines their outputs for better accuracy and robustness.

- **Key Idea:** Train multiple decision trees on different random subsets of data and features, then aggregate their predictions.
 - Classification → Majority vote
 - Regression → Average of outputs
- Helps reduce **overfitting** that often occurs with individual decision trees.

Algorithm Steps:

1. Draw random samples from the dataset (bootstrapping).
2. Train a decision tree on each sample, but at each split, consider only a random subset of features.
3. Aggregate results from all trees.

Support Vector Machines (SVM): Theory + From-Scratch & Library Implementations

Theory

Support Vector Machines (SVM) is a **supervised learning algorithm** used mainly for **classification**, but also for regression tasks (SVR).

- **Key Idea:** Find the best **hyperplane** that separates data points of different classes with the **maximum margin** (distance between nearest data points from each class and the hyperplane).
- The data points that lie closest to the hyperplane are called **support vectors**.
- Works well in high-dimensional spaces using **kernel tricks** (e.g., linear, polynomial, RBF kernels).

Algorithm Steps (Classification):

1. Map data into high-dimensional feature space (possibly using kernels).
2. Find a separating hyperplane that maximizes margin.
3. Predict class based on which side of the hyperplane the point lies.

k-Nearest Neighbors (k-NN): Theory + From-Scratch & Library Implementations

Theory

k-Nearest Neighbors (k-NN) is a **supervised learning algorithm** used for both **classification** and **regression**.

- **Key Idea:** To classify a data point, look at the **k nearest data points** (neighbors) in the training set and assign the majority label (classification) or average value (regression).
- It is a **lazy learner**: no explicit training, predictions are made directly based on the dataset.
- Distance metric is crucial (commonly **Euclidean distance**).

Algorithm Steps:

1. Choose a value for **k**.
2. Compute the distance between the query point and all training points.
3. Select the **k closest neighbors**.
4. For classification → majority vote, for regression → average.

Neural Networks: Theory + From-Scratch & Library Implementations

Theory

Neural Networks are a class of **supervised learning models** inspired by the human brain. They consist of layers of **neurons** connected by **weights**.

- **Key Idea:** Learn complex, non-linear mappings from inputs to outputs through multiple layers.
- **Components:**
 - **Input layer:** Features
 - **Hidden layers:** Intermediate computations with activation functions (ReLU, Sigmoid, Tanh)
 - **Output layer:** Predictions (classification → softmax, regression → linear)
- Training is done using **backpropagation** and **gradient descent** to minimize a loss function.

K-Means Clustering: Theory + From-Scratch & Library Implementations

Theory

- K-Means is an **unsupervised clustering algorithm**.
- Goal: group data into **K clusters** where each point belongs to the cluster with the nearest centroid.
- Iterative process:
 1. Initialize K centroids randomly.
 2. Assign each data point to the nearest centroid.
 3. Recalculate centroids as mean of points in cluster.
 4. Repeat until convergence (centroids don't change).

DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Theory + From-Scratch & Library Implementations

Theory

DBSCAN is an **unsupervised clustering algorithm** that groups together points that are close to each other (high density regions) and marks points in low-density regions as **outliers (noise)**. Unlike K-Means, DBSCAN does not require the number of clusters in advance.

- **Key Parameters:**
 - `eps` : The maximum distance between two samples for them to be considered neighbors.
 - `min_samples` : The minimum number of points required to form a dense region (cluster).
- **Types of Points:**
 1. **Core Point:** Has at least `min_samples` points within distance `eps`.
 2. **Border Point:** Not a core point, but within `eps` of a core point.
 3. **Noise Point:** Neither core nor border (considered outlier).
- **Algorithm Steps:**
 1. Pick an unvisited point.
 2. If it's a core point, form a cluster including all points density-reachable from it.
 3. If it's a border point, assign it to the nearest cluster.
 4. If it's noise, mark as outlier.

Apriori Algorithm: Theory + From-Scratch & Library Implementations

Theory

The Apriori Algorithm is an **unsupervised learning algorithm** used for **association rule mining**. It helps discover interesting relationships (association rules) among items in large datasets, often used in **market basket analysis**.

- **Key Concepts:**
 - **Itemset:** A collection of items (e.g., {milk, bread}).
 - **Support:** Fraction of transactions containing an itemset.
 - **Confidence:** Likelihood that if a customer buys item A, they also buy item B.
 - **Lift:** How much more often item A and B occur together than expected if independent.
- **Algorithm Steps:**
 1. Generate all possible itemsets.
 2. Keep only itemsets with support $\geq$ `min_support`.
 3. Use frequent itemsets to generate larger candidate itemsets.
 4. Generate association rules from frequent itemsets using confidence $\geq$ `min_confidence`.